

LAPORAN PRAKTIKUM STRUKTUR DATA

QUEUE

Disusun Oleh:

Endy Pardilian 2511531017

Dosen Pengampu:

Wahyudi. Dr., S.T,M.T

Asisten Pratikum:

M Galid Avero



DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

PADANG

2026

KATA PENGANTAR

Puji syukur penulis panjatkan kehadiran Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya laporan praktikum ini dapat diselesaikan dengan baik. Laporan ini disusun sebagai salah satu tugas pada mata kuliah Praktikum Struktur Data dengan topik pembahasan mengenai struktur data Queue (antrian). Dalam laporan ini dibahas implementasi queue menggunakan beberapa metode, seperti array dan LinkedList, serta penerapannya dalam studi kasus sederhana berupa sistem antrian loket, termasuk operasi dasar seperti enqueue, dequeue, dan fitur tambahan seperti reverse data. Penulis menyadari bahwa laporan ini masih memiliki kekurangan, sehingga kritik dan saran yang membangun sangat diharapkan demi perbaikan di masa yang akan datang, serta semoga laporan ini dapat memberikan manfaat dan menambah pemahaman mengenai struktur data queue.

Padang, 2026

Penulis

DAFTAR ISI

KATA PENGANTAR.....	ii
DAFTAR ISI.....	iii
BAB I PENDAHULUAN.....	1
1.1 Latar Belakang.....	1
1.2 Tujuan.....	1
1.3 Manfaat.....	1
BAB II PEMBAHASAN.....	2
2.1 QueueArray_2511531017.....	2
2.2 QueuearrayDriver_2511531017.....	5
2.3 QueueLinkedList_2511531017.....	6
2.4 IterasiQueue_2511531017.....	8
2.5 ReverseData_2511531017.....	9
TUGAS.....	11
2.6 Queue_2511531017.....	11
2.7 AntrianLoket_2511531017.....	14
BAB III KESIMPULAN.....	18
3.1 Kesimpulan.....	18
3.2 Saran.....	18
DAFTAR PUSTAKA.....	19

BAB I PENDAHULUAN

1.1 Latar Belakang

Struktur data merupakan salah satu konsep dasar dalam pemrograman yang berperan penting dalam pengelolaan dan penyimpanan data secara efisien. Salah satu struktur data yang sering digunakan adalah queue (antrian), yang menerapkan prinsip FIFO (First In First Out), di mana data yang pertama masuk akan menjadi data pertama yang keluar. Konsep ini banyak diterapkan dalam kehidupan sehari-hari, seperti pada sistem antrian di loket pelayanan, rumah sakit, maupun proses pengolahan data dalam sistem komputer. Oleh karena itu, pemahaman mengenai queue menjadi hal yang penting bagi mahasiswa dalam mempelajari struktur data. Selain itu, dengan memahami queue, mahasiswa dapat mengembangkan logika pemrograman dalam menyelesaikan permasalahan yang berkaitan dengan pengurutan dan pengelolaan data. Penerapan queue juga membantu dalam memahami bagaimana sistem bekerja secara terstruktur dan sistematis. Pada praktikum ini, dilakukan implementasi queue menggunakan berbagai pendekatan, seperti array dan LinkedList, serta penerapannya dalam studi kasus sederhana untuk memahami cara kerja dan pengelolaan data dalam antrian secara lebih mendalam.

1.2 Tujuan

1. Memahami konsep dasar struktur data queue (antrian) beserta prinsip FIFO (First In First Out).
2. Mampu mengimplementasikan queue menggunakan berbagai metode, seperti array dan LinkedList.
3. Mengetahui cara kerja operasi dasar queue seperti enqueue, dequeue, serta pengembangan fitur seperti reverse data..

1.3 Manfaat

1. Membantu meningkatkan pemahaman mahasiswa terhadap struktur data queue dalam pemrograman.
2. Melatih kemampuan dalam mengelola dan memanipulasi data secara terstruktur dan sistematis.
3. Memberikan gambaran penerapan queue dalam kehidupan nyata, seperti sistem antrian pada layanan publik.

BAB II PEMBAHASAN

2.1 QueueArray_2511531017

```
package pekan4_2511531017;

public class QueueArray_2511531017 {
    int front, rear, size;
    int capacity;
    int array[];

    public QueueArray_2511531017(int capacity) {
        this.capacity = capacity;
        front = this.size = 0;
        rear = capacity - 1;
        array = new int[this.capacity];
    }

    boolean isFull_1017(QueueArray_2511531017 queue) {
        return (queue.size == queue.capacity);
    }

    boolean isEmpty_1017(QueueArray_2511531017 queue) {
        return (queue.size == 0);
    }

    void enqueue_1017 (int item) {
        if (isFull_1017(this))
            return;
        this.rear = (this.rear + 1) % this.capacity;
        this.array[this.rear]=item;
        this.size = this.size + 1;
        System.out.println(item + " enqueue to queue");
    }

    int dequeue_1017() {
        if (isEmpty_1017(this))
            return Integer.MIN_VALUE;
        int item = this.array[this.front];
        this.front = (this.front + 1) % this.capacity;
        this.size = this.size - 1;
        return item;
    }

    int front_1017() {
        if (isEmpty_1017(this))
            return Integer.MIN_VALUE;
        return this.array[this.front];
    }

    int rear_1017() {
        if (isEmpty_1017(this))
            return Integer.MIN_VALUE;
        return this.array[this.rear];
    }

    void display_1017() {
```

```

        int i;
        if (front == rear) {
            System.out.printf("\nAntrian Kosong\n");
            return;
        }
        for (i = front; i < rear; i++) {
            System.out.printf(" %d <-- ", array[i]);
        }
        return;
    }
}

```

Kode 2.1

Penjelasan:

```

public QueueArray_2511531017(int capacity) {
    this.capacity = capacity;
    front = this.size = 0;
    rear = capacity - 1;
    array = new int[this.capacity];
}

```

Method ini merupakan constructor yang digunakan untuk menginisialisasi objek queue. Parameter capacity berfungsi untuk menentukan kapasitas maksimal antrian. Variabel front dan size di-set ke 0, yang berarti antrian masih kosong di awal. Sedangkan rear di-set ke capacity - 1 agar saat elemen pertama dimasukkan, posisi rear akan kembali ke indeks 0 (karena menggunakan konsep circular queue). Selain itu, array dibuat dengan ukuran sesuai kapasitas untuk menyimpan elemen-elemen queue.

```

boolean isFull_1017(QueueArray_2511531017 queue) {
    return (queue.size == queue.capacity);
}

```

Method ini digunakan untuk mengecek apakah antrian sudah penuh atau belum. Kondisi penuh terjadi ketika jumlah elemen (size) sama dengan kapasitas maksimum (capacity). Jika penuh, method akan mengembalikan nilai true, jika tidak maka false.

```

boolean isEmpty_1017(QueueArray_2511531017 queue) {
    return (queue.size == 0);
}

```

Method ini berfungsi untuk mengecek apakah antrian kosong. Jika size bernilai 0, maka queue tidak memiliki elemen. Method ini penting untuk mencegah error saat melakukan penghapusan data (dequeue).

```

void enqueue_1017 (int item) {
    if (isFull_1017(this))
        return;
    this.rear = (this.rear + 1) % this.capacity;
    this.array[this.rear]=item;
    this.size = this.size + 1;
    System.out.println(item + " enqueue to queue");
}

```

Method ini digunakan untuk menambahkan elemen ke dalam antrian. Pertama, dicek apakah queue penuh menggunakan isFull. Jika penuh, proses dihentikan. Jika tidak penuh rear digeser ke posisi berikutnya, lalu nilai item dimasukkan ke array pada indeks rear dan size ditambah 1. Terakhir, program menampilkan pesan bahwa data berhasil dimasukkan.

```
int dequeue_1017() {
    if (isEmpty_1017(this))
        return Integer.MIN_VALUE;
    int item = this.array[this.front];
    this.front = (this.front + 1) % this.capacity;
    this.size = this.size - 1;
    return item;
}
```

Method ini berfungsi untuk menghapus elemen dari antrian (FIFO). Pertama cek apakah queue kosong. Jika kosong, mengembalikan Integer.MIN_VALUE sebagai tanda error. Jika tidak, ambil nilai di posisi front lalu geser front ke indeks berikutnya (circular), kurangi size dan kembalikan nilai yang dihapus.

```
int front_1017() {
    if (isEmpty_1017(this))
        return Integer.MIN_VALUE;
    return this.array[this.front];
}
```

Method ini digunakan untuk melihat elemen paling depan tanpa menghapusnya. Jika queue kosong, akan mengembalikan nilai minimum integer sebagai indikator tidak ada data.

```
int rear_1017() {
    if (isEmpty_1017(this))
        return Integer.MIN_VALUE;
    return this.array[this.rear];
}
```

Method ini berfungsi untuk melihat elemen paling belakang dari antrian tanpa menghapusnya.

```
void display_1017() {
    int i;
    if (front == rear) {
        System.out.printf("\nAntrian Kosong\n");
        return;
    }
    for (i = front; i < rear; i++) {
        System.out.printf(" %d <-- ", array[i]);
    }
    return;
}
```

Method ini digunakan untuk menampilkan isi antrian dari depan ke belakang. Jika front = rear, maka dianggap antrian kosong dan akan menampilkan pesan "Antrian Kosong". Jika tidak kosong, method akan melakukan perulangan dari indeks front sampai rear untuk mencetak setiap elemen dalam queue.

Class QueueArray_2511531017 merupakan implementasi dari struktur data queue menggunakan array sebagai media penyimpanan. Class ini menerapkan konsep circular queue, sehingga memungkinkan penggunaan memori array secara optimal dengan memanfaatkan kembali indeks yang telah kosong. Class ini menyediakan berbagai method untuk mendukung operasi dasar queue, seperti enqueue untuk menambahkan elemen, dequeue untuk menghapus elemen, serta method lain untuk pengecekan kondisi queue dan menampilkan isi antrian.

2.2 QueuearrayDriver_2511531017

```
package pekan4_2511531017;

public class QueuearrayDriver_2511531017 {

    public static void main(String[] args) {
        // TODO Auto-generated method stub
        QueueArray_2511531017 queue = new QueueArray_2511531017(1000);
        queue.enqueue_1017(10);
        queue.enqueue_1017(20);
        queue.enqueue_1017(30);
        queue.enqueue_1017(40);
        System.out.println("Item di depan " + queue.front_1017());
        System.out.println("Item paling belakang " + queue.rear_1017());
        System.out.println("tampilan queue");
        queue.display_1017();
        System.out.println();
        System.out.println(queue.dequeue_1017() + " dihapus dari queue");
        System.out.println("Item di depan: " + queue.front_1017());
        System.out.println("Item paling belakang: " + queue.rear_1017());
        System.out.println("tampilan queue setelah satu data dihapus");
        queue.display_1017();

    }

}
```

Kode 2.2

Penjelasan:

```
QueueArray_2511531017 queue = new QueueArray_2511531017(1000);
```

Baris ini digunakan untuk membuat objek queue dengan kapasitas maksimal 1000 elemen. Objek ini akan digunakan untuk melakukan berbagai operasi antrian.

```
queue.enqueue_1017(10);
queue.enqueue_1017(20);
queue.enqueue_1017(30);
queue.enqueue_1017(40);
```

Baris-baris ini menambahkan data ke dalam queue secara berurutan, yaitu 10, 20, 30, dan 40. Karena queue menggunakan konsep FIFO, maka urutan masuk akan sama dengan urutan keluar.

```
System.out.println("Item di depan " + queue.front_1017());
System.out.println("Item paling belakang " + queue.rear_1017());
```

Digunakan untuk menampilkan elemen paling depan (yang pertama masuk) dan elemen paling belakang (yang terakhir masuk)

```
System.out.println("tampilan queue");
queue.display_1017();
```

Menampilkan seluruh isi antrian dari depan ke belakang menggunakan method display_1017().


```
System.out.println(queue.dequeue_1017() + " dihapus dari queue");
```

Menghapus elemen paling depan dari queue (yaitu 10) sesuai prinsip FIFO, lalu menampilkannya ke layar.

```
System.out.println("Item di depan: " + queue.front_1017());  
System.out.println("Item paling belakang: " + queue.rear_1017());
```

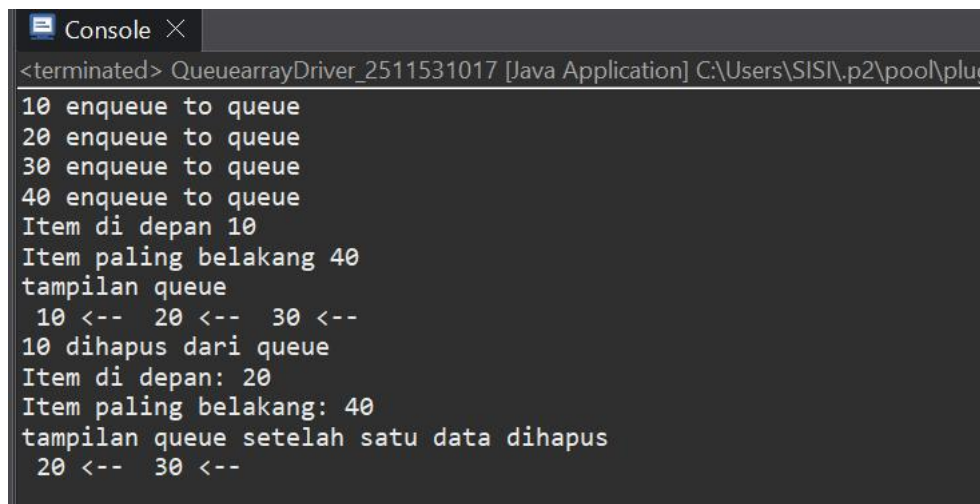
Setelah satu elemen dihapus, elemen depan berubah (dari 10 menjadi 20), elemen belakang tetap (40)

```
System.out.println("tampilan queue setelah satu data dihapus");  
queue.display_1017();
```

Menampilkan isi queue terbaru setelah proses dequeue, sehingga terlihat bahwa satu elemen sudah hilang dari antrian.

Class ini berfungsi sebagai program utama (driver) untuk menjalankan dan menguji class QueueArray_2511531017. Di dalamnya terdapat method main yang digunakan untuk mensimulasikan operasi-operasi pada queue.

Output:



```
<terminated> QueuearrayDriver_2511531017 [Java Application] C:\Users\SISI\p2\pool\plug  
10 enqueue to queue  
20 enqueue to queue  
30 enqueue to queue  
40 enqueue to queue  
Item di depan 10  
Item paling belakang 40  
tampilan queue  
10 <-- 20 <-- 30 <--  
10 dihapus dari queue  
Item di depan: 20  
Item paling belakang: 40  
tampilan queue setelah satu data dihapus  
20 <-- 30 <--
```

Gambar 2.2

2.3 QueueLinkedList_2511531017

```
package pekan4_2511531017;  
  
import java.util.*;  
  
public class QueueLinkedList_2511531017 {  
  
    public static void main(String[] args) {  
        // TODO Auto-generated method stub  
        Queue<Integer> q = new LinkedList<>();  
        for (int i = 0; i<6; i++)  
            q.add(i);  
        System.out.println("Elemen Antrian " + q);  
        int hapus = q.remove();  
        System.out.println("Hapus elemen " + hapus);  
    }  
}
```

```

        System.out.println(q);
        int depan = q.peek();
        System.out.println("Kepala Antrian = " + depan);
        int banyak = q.size();
        System.out.println("Size Antriam =" + banyak);

    }
}

```

Kode 2.3

Penjelasan:

```

for (int i = 0; i<6; i++)
    q.add(i);

```

Perulangan ini menambahkan elemen ke queue dari 0 sampai 5. Method add() berfungsi seperti enqueue, yaitu menambahkan data ke bagian belakang antrian.

```

System.out.println("Elemen Antrian " + q);

```

Menampilkan seluruh isi queue. Karena menggunakan LinkedList, isi antrian langsung bisa ditampilkan dalam bentuk list.

```

int hapus = q.remove();
System.out.println("Hapus elemen " + hapus);

```

Method remove() digunakan untuk menghapus elemen paling depan dari queue (FIFO). Nilai yang dihapus disimpan dalam variabel hapus lalu ditampilkan.

```

System.out.println(q);

```

Menampilkan kondisi queue setelah satu elemen dihapus.

```

int depan = q.peek();
System.out.println("Kepala Antrian = " + depan);

```

Method peek() digunakan untuk melihat elemen paling depan tanpa menghapusnya.

```

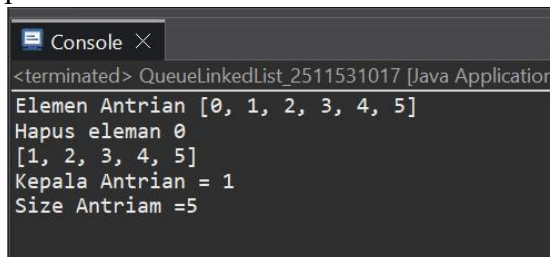
int banyak = q.size();
System.out.println("Size Antriam =" + banyak);

```

Method size() digunakan untuk mengetahui jumlah elemen yang ada di dalam queue saat ini.

Program ini menunjukkan penggunaan queue dengan memanfaatkan LinkedList bawaan Java. Operasi seperti penambahan, penghapusan, melihat elemen depan, dan menghitung jumlah elemen dapat dilakukan dengan lebih sederhana tanpa harus mengimplementasikan struktur data secara manual seperti pada queue berbasis array.

Output:



```

<terminated> QueueLinkedList_2511531017 [Java Application]
Elemen Antrian [0, 1, 2, 3, 4, 5]
Hapus elemen 0
[1, 2, 3, 4, 5]
Kepala Antrian = 1
Size Antriam =5

```

Gambar 2.3

2.4 IterasiQueue_2511531017

```
package pekan4_2511531017;
import java.util.*;
public class IterasiQueue_2511531017 {

    public static void main(String[] args) {
        // TODO Auto-generated method stub
        Queue<String> q = new LinkedList<>();

        q.add("Praktikum");
        q.add("Struktur");
        q.add("Data");
        q.add("Dan");
        q.add("Algoritma");
        Iterator<String> iterator = q.iterator();
        while (iterator.hasNext()) {
            System.out.print(iterator.next() + " ");
        }
    }
}
```

Kode 2.4

Penjelasan:

```
q.add("Praktikum");
q.add("Struktur");
q.add("Data");
q.add("Dan");
q.add("Algoritma");
```

Menambahkan beberapa elemen string ke dalam queue. Elemen-elemen ini akan tersimpan sesuai urutan penambahan (FIFO).

```
Iterator<String> iterator = q.iterator();
```

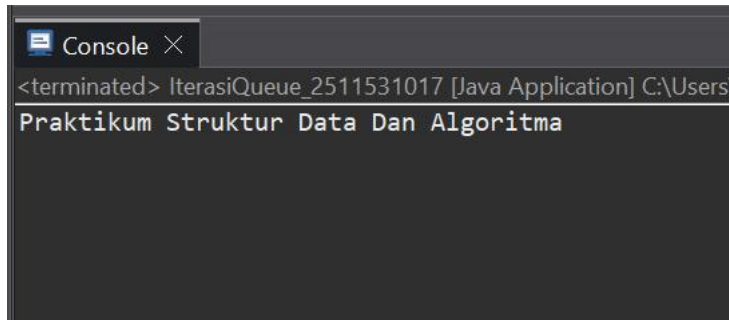
Membuat objek Iterator untuk mengakses elemen-elemen dalam queue. Iterator digunakan untuk menelusuri isi struktur data tanpa harus menggunakan indeks.

```
while (iterator.hasNext()) {
    System.out.print(iterator.next() + " ");
}
```

Perulangan ini digunakan untuk menampilkan semua elemen dalam queue. hasNext(), untuk mengecek apakah masih ada elemen berikutnya, next(), untuk mengambil elemen berikutnya. Setiap elemen akan dicetak hingga seluruh isi queue selesai ditampilkan.

Program ini menunjukkan cara mengakses seluruh elemen dalam queue menggunakan Iterator. Dengan metode ini, elemen dapat ditelusuri secara berurutan tanpa perlu menghapus data dari queue, sehingga struktur data tetap utuh.

Output:



Gambar 2.4

2.5 ReverseData_2511531017

```
package pekan4_2511531017;
import java.util.*;
public class ReverseData_2511531017 {

    public static void main(String[] args) {
        // TODO Auto-generated method stub
        Queue<Integer> q = new LinkedList<Integer>();
        q.add(1);
        q.add(2);
        q.add(3);
        System.out.println("sebelum reverse" + q);
        Stack<Integer> s = new Stack<Integer>();
        while (!q.isEmpty()) {
            s.push(q.remove());
        }
        while (!s.isEmpty()) {
            q.add(s.pop());
        }
        System.out.println("sesudah reverse= " + q);
    }
}
```

Kode 2.5

Penjelasan:

```
Queue<Integer> q = new LinkedList<Integer>();
```

Membuat queue bertipe Integer menggunakan LinkedList sebagai implementasi.

```
q.add(1);
q.add(2);
q.add(3);
```

Menambahkan elemen ke dalam queue dengan urutan 1 - 2 - 3

```
System.out.println("sebelum reverse" + q);
```

Menampilkan isi queue sebelum dilakukan proses pembalikan.

```
Stack<Integer> s = new Stack<Integer>();
```

Membuat stack yang akan digunakan sebagai alat bantu untuk membalik urutan data.

```
while (!q.isEmpty()) {  
    s.push(q.remove());
```

Semua elemen dalam queue dipindahkan ke stack. `q.remove()`, untuk ambil dari depan queue, `s.push()`, untuk masuk ke stack. Karena stack bersifat LIFO, urutan data otomatis terbalik.

```
while (!s.isEmpty()) {  
    q.add(s.pop());
```

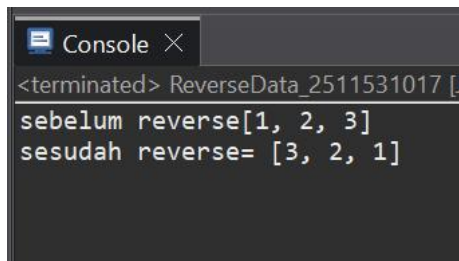
Data dari stack dikembalikan ke queue. `s.pop()`, untuk ambil dari atas stack, `q.add()`, untuk masuk ke queue. Hasilnya, urutan queue menjadi terbalik dari sebelumnya.

```
System.out.println("sesudah reverse= " + q);
```

Menampilkan hasil akhir queue setelah dilakukan reverse.

Program ini menunjukkan bahwa pembalikan data pada queue dapat dilakukan dengan bantuan stack. Dengan memanfaatkan perbedaan prinsip FIFO pada queue dan LIFO pada stack, urutan elemen dalam queue dapat dibalik secara efektif.

Output:



```
<terminated> ReverseData_2511531017 [  
sebelum reverse[1, 2, 3]  
sesudah reverse= [3, 2, 1]
```

Gambar 2.5

TUGAS

2.6 Queue_2511531017

```
package pekan4_2511531017;

public class Queue_2511531017 {
    int front, rear, max;
    String queue[];

    public Queue_2511531017(int max) {
        this.max = max;
        queue = new String[max];
        front = -1;
        rear = -1;
    }

    public boolean isEmpty_1017() {
        return (front == -1);
    }

    public boolean isFull_1017() {
        return (rear == max - 1);
    }

    public void enqueue_1017(String data) {
        if (isFull_1017()) {
            System.out.println("Antrian penuh!");
        } else {
            if (isEmpty_1017()) {
                front = 0;
            }
            rear++;
            queue[rear] = data;
            System.out.println("Data berhasil ditambahkan ke antrian");
        }
    }

    public void dequeue_1017() {
        if (isEmpty_1017()) {
            System.out.println("Antrian kosong!");
        } else {
            System.out.println(queue[front] + " telah dilayani");
            front++;
            if (front > rear) {
                front = rear = -1;
            }
        }
    }

    public void display_1017() {
        if (isEmpty_1017()) {
            System.out.println("Antrian kosong!");
        } else {
            System.out.println("Isi antrian:");
            int no = 1;
        }
    }
}
```

```

        for (int i = front; i <= rear; i++) {
            System.out.println(no + ". " + queue[i]);
            no++;
        }
    }

    public void reverse_1017() {
        if (isEmpty_1017()) {
            System.out.println("Antrian kosong!");
        } else {
            int start = front;
            int end = rear;
            while (start < end) {
                String temp = queue[start];
                queue[start] = queue[end];
                queue[end] = temp;
                start++;
                end--;
            }
        }
    }
}

```

Kode 2.6

Penjelasan:

```

public Queue_2511531017(int max) {
    this.max = max;
    queue = new String[max];
    front = -1;
    rear = -1;
}

```

Method ini merupakan constructor yang digunakan untuk menginisialisasi queue dengan kapasitas maksimum tertentu. Array queue dibuat sesuai ukuran max untuk menyimpan data bertipe String. Nilai front dan rear di-set ke -1 sebagai tanda bahwa antrian masih kosong.

```

public boolean isEmpty_1017() {
    return (front == -1);
}

```

Method ini berfungsi untuk mengecek apakah queue kosong. Jika front bernilai -1, maka antrian belum memiliki elemen dan method akan mengembalikan nilai true.

```

public boolean isFull_1017() {
    return (rear == max - 1);
}

```

Method ini digunakan untuk mengecek apakah queue sudah penuh. Jika rear sudah berada di indeks terakhir array (max - 1), maka queue tidak bisa ditambahkan elemen lagi.

```

public void enqueue_1017(String data) {
    if (isFull_1017()) {
        System.out.println("Antrian penuh!");
    } else {
        if (isEmpty_1017()) {
            front = 0;
        }
        rear++;
    }
}

```

```
queue[rear] = data;
System.out.println("Data berhasil ditambahkan ke antrian");
```

Method ini berfungsi untuk menambahkan data ke dalam antrian. Pertama, dilakukan pengecekan apakah queue penuh. Jika tidak, dan queue dalam kondisi kosong, maka front di-set ke 0. Setelah itu, rear ditambah satu dan data dimasukkan ke array. Terakhir, ditampilkan pesan bahwa data berhasil ditambahkan.

```
public void dequeue_1017() {
    if (isEmpty_1017()) {
        System.out.println("Antrian kosong!");
    } else {
        System.out.println(queue[front] + " telah dilayani");
        front++;
        if (front > rear) {
            front = rear = -1;
        }
    }
}
```

Method ini digunakan untuk menghapus elemen paling depan dari queue. Jika antrian kosong, akan ditampilkan pesan error. Jika tidak, data pada posisi front akan ditampilkan sebagai data yang dihapus, kemudian front digeser ke kanan. Jika seluruh data sudah habis, maka front dan rear akan di-reset ke -1.

```
public void display_1017() {
    if (isEmpty_1017()) {
        System.out.println("Antrian kosong!");
    } else {
        System.out.println("Isi antrian:");
        int no = 1;
        for (int i = front; i <= rear; i++) {
            System.out.println(no + ". " + queue[i]);
            no++;
        }
    }
}
```

Method ini berfungsi untuk menampilkan seluruh isi antrian. Jika queue kosong, akan ditampilkan pesan bahwa antrian kosong. Jika tidak, data ditampilkan dari front sampai rear menggunakan perulangan, serta diberi nomor urut agar lebih rapi.

```
public void reverse_1017() {
    if (isEmpty_1017()) {
        System.out.println("Antrian kosong!");
    } else {
        int start = front;
        int end = rear;
        while (start < end) {
            String temp = queue[start];
            queue[start] = queue[end];
            queue[end] = temp;
            start++;
            end--;
        }
    }
}
```

Method ini digunakan untuk membalik urutan data dalam queue tanpa menggunakan bantuan stack. Proses dilakukan dengan menukar elemen dari depan (start) dan belakang (end) secara bertahap hingga bertemu di tengah.

2.7 AntrianLoket_2511531017

```
package pekan4_2511531017;

import java.util.*;

public class AntrianLoket_2511531017 {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        Queue_2511531017 antrian = new Queue_2511531017(10);

        int pilihan;
        do {
            System.out.println("\n=== PROGRAM ANTRIAN LOKET ===");
            System.out.println("1. Tambah Antrian");
            System.out.println("2. Hapus Antrian");
            System.out.println("3. Tampilkan Antrian");
            System.out.println("4. Reverse");
            System.out.println("5. Keluar");
            System.out.print("Pilih menu: ");
            pilihan = input.nextInt();
            input.nextLine();

            switch (pilihan) {
                case 1:
                    System.out.print("Masukkan nama pelanggan: ");
                    String nama = input.nextLine();
                    antrian.enqueue_1017(nama);
                    break;
                case 2:
                    antrian.dequeue_1017();
                    break;
                case 3:
                    antrian.display_1017();
                    break;
                case 4:
                    antrian.reverse_1017();
                    antrian.display_1017();
                    break;
                case 5:
                    System.out.println("Program selesai");
                    break;
                default:
                    System.out.println("Pilihan tidak valid!");
            }
        } while (pilihan != 5);

        input.close();
    }
}
```

Kode 2.7

Penjelasan:

```
Queue_2511531017 antrian = new Queue_2511531017(10);
```

Class dengan kapasitas maksimal 10 data untuk menyimpan nama pelanggan.

```
int pilihan;
do {
    System.out.println("\n=== PROGRAM ANTRIAN LOKET ===");
    System.out.println("1. Tambah Antrian");
    System.out.println("2. Hapus Antrian");
    System.out.println("3. Tampilkan Antrian");
    System.out.println("4. Reverse");
    System.out.println("5. Keluar");
    System.out.print("Pilih menu: ");
    pilihan = input.nextInt();
    input.nextLine();
}
```

Perulangan do-while digunakan agar program terus berjalan dan menampilkan menu berulang kali sampai pengguna memilih keluar, lalu menampilkan daftar menu yang dapat dipilih oleh pengguna untuk melakukan operasi pada antrian.

```
switch (pilihan) {
    case 1:
        System.out.print("Masukkan nama pelanggan: ");
        String nama = input.nextLine();
        antrian.enqueue_1017(nama);
        break;
```

Digunakan untuk menentukan aksi yang akan dilakukan berdasarkan pilihan pengguna. Case 1 digunakan untuk menambahkan data pelanggan ke dalam antrian melalui input pengguna.

```
    case 2:
        antrian.dequeue_1017();
        break;
```

Case 2 digunakan untuk menghapus data paling depan dari antrian sesuai prinsip FIFO.

```
    case 3:
        antrian.display_1017();
        break;
```

Case 3 menampilkan seluruh isi antrian yang ada saat ini.

```
    case 4:
        antrian.reverse_1017();
        antrian.display_1017();
        break;
```

Case 4 digunakan untuk membalik urutan antrian, kemudian langsung menampilkan hasilnya.

```
    case 5:
        System.out.println("Program selesai");
        break;
```

Case 5 menghentikan program ketika pengguna memilih menu keluar.

```
default:  
    System.out.println("Pilihan tidak valid!");
```

Menampilkan pesan jika input yang dimasukkan tidak sesuai dengan pilihan menu.

```
while (pilihan != 5);  
  
    input.close();
```

Program akan terus berjalan selama pengguna belum memilih opsi keluar.

Program AntrianLoket_2511531017 berfungsi sebagai simulasi sistem antrian loket yang memanfaatkan struktur data queue. Program ini memungkinkan pengguna untuk menambahkan pelanggan ke dalam antrian, menghapus pelanggan yang telah dilayani, menampilkan daftar antrian, serta membalik urutan antrian melalui menu interaktif. Dengan menggunakan class Queue_2511531017, program ini menerapkan prinsip FIFO (First In First Out) dalam pengelolaan antrian secara sederhana.

Output:

```
=== PROGRAM ANTRIAN LOKET ===  
1. Tambah Antrian  
2. Hapus Antrian  
3. Tampilkan Antrian  
4. Reverse  
5. Keluar  
Pilih menu: 1  
Masukkan nama pelanggan: endy  
Data berhasil ditambahkan ke antrian  
  
=== PROGRAM ANTRIAN LOKET ===  
1. Tambah Antrian  
2. Hapus Antrian  
3. Tampilkan Antrian  
4. Reverse  
5. Keluar  
Pilih menu: 1  
Masukkan nama pelanggan: pardilian  
Data berhasil ditambahkan ke antrian  
  
=== PROGRAM ANTRIAN LOKET ===  
1. Tambah Antrian  
2. Hapus Antrian  
3. Tampilkan Antrian  
4. Reverse  
5. Keluar  
Pilih menu: 3  
Isi antrian:  
1. endy  
2. pardilian
```

```
=== PROGRAM ANTRIAN LOKET ===  
1. Tambah Antrian  
2. Hapus Antrian  
3. Tampilkan Antrian  
4. Reverse  
5. Keluar  
Pilih menu: 4  
Isi antrian:  
1. pardilian  
2. endy
```

```
=== PROGRAM ANTRIAN LOKET ===  
1. Tambah Antrian  
2. Hapus Antrian  
3. Tampilkan Antrian  
4. Reverse  
5. Keluar  
Pilih menu: 2  
pardilian telah dilayani
```

```
=== PROGRAM ANTRIAN LOKET ===  
1. Tambah Antrian  
2. Hapus Antrian  
3. Tampilkan Antrian  
4. Reverse  
5. Keluar  
Pilih menu: 3  
Isi antrian:  
1. endy
```

```
=== PROGRAM ANTRIAN LOKET ===  
1. Tambah Antrian  
2. Hapus Antrian  
3. Tampilkan Antrian  
4. Reverse  
5. Keluar  
Pilih menu: 5  
Program selesai
```

Gambar 2.7

BAB III KESIMPULAN

3.1 Kesimpulan

Struktur data queue merupakan salah satu konsep penting dalam pemrograman yang menerapkan prinsip FIFO (First In First Out), di mana data yang pertama masuk akan menjadi data pertama yang keluar. Melalui praktikum ini, dapat dipahami bagaimana queue diimplementasikan menggunakan berbagai metode, seperti array dan LinkedList, serta bagaimana operasi dasar seperti enqueue, dequeue, dan pengecekan kondisi antrian bekerja dalam pengelolaan data. Selain itu, penggunaan queue membantu dalam memahami alur pengolahan data yang berurutan dan terstruktur. Hal ini juga melatih kemampuan berpikir logis dalam menyusun program yang efisien.

Selain itu, praktikum ini juga menunjukkan bahwa queue dapat dikembangkan dengan fitur tambahan seperti reverse data dan diterapkan dalam studi kasus sederhana seperti sistem antrian loket. Dengan demikian, pemahaman terhadap queue tidak hanya bersifat teoritis, tetapi juga dapat diaplikasikan secara langsung dalam penyelesaian permasalahan nyata secara terstruktur dan efisien. Penerapan ini memberikan gambaran nyata bagaimana struktur data digunakan dalam kehidupan sehari-hari. Hal ini juga meningkatkan kemampuan mahasiswa dalam menghubungkan konsep teori dengan praktik di lapangan.

3.2 Saran

Sebagai saran, sebaiknya penjelasan materi saat praktikum bisa lebih detail dan perlahan, supaya mahasiswa yang belum terlalu paham coding bisa mengikuti dengan baik.

DAFTAR PUSTAKA

- [1] Oracle, “Queue Interface (Java Platform SE Documentation),” [Online]. Available: <https://docs.oracle.com/javase/8/docs/api/java/util/Queue.html>